

Autoescalamento Horizontal de uma Aplicação Web em Kubernetes: comparação entre Minikube e Amazon EKS

Mario Marszalek

PSI5120 – Tópicos em Computação em Nuvem
Escola Politécnica – Universidade de São Paulo
NUSP: 11910490 – mariomarszalek@usp.br

Resumo—Este trabalho implementa e avalia o autoescalamento horizontal de uma aplicação web em Kubernetes. Foram feitas duas implantações do mesmo sistema: uma local, usando Minikube, e outra gerenciada, usando Amazon EKS. A aplicação em PHP recebe uma carga de CPU controlada por parâmetro; o Horizontal Pod Autoscaler (HPA) usa CPU média, com alvo de 50%, para variar entre uma e cinco réplicas. No EKS, a utilização registrada passou de 1% para 498% do *request* de CPU, enquanto a meta do HPA era 50%; a captura do pico mostra quatro réplicas ativas. O experimento também mostra a diferença prática entre o ambiente local, rápido para iterar, e o ambiente gerenciado, que exige mais configuração, credenciais e cuidado com custos.

Index Terms—Kubernetes, Horizontal Pod Autoscaler, HPA, Minikube, Amazon EKS, computação em nuvem.

I. INTRODUÇÃO

Aplicações web têm demanda variável. Manter um número fixo de processos pode deixar o sistema lento em picos ou gastar recurso à toa quando não há tráfego. Em Kubernetes, o Horizontal Pod Autoscaler ajusta a quantidade de réplicas de um recurso escalável, como um *Deployment*, a partir de métricas [1]. Neste trabalho foi usado o HPA baseado em utilização média de CPU.

O objetivo não foi montar uma aplicação complexa. A ideia foi montar um caso pequeno que desse para medir: uma aplicação web, um *Service*, um *Deployment*, um gerador de carga e o HPA. O mesmo desenho foi executado localmente no Minikube e na nuvem no Amazon EKS. Assim, foi possível observar tanto o comportamento do escalonamento quanto o custo operacional de subir a infraestrutura.

II. ARQUITETURA DA SOLUÇÃO

A Figura 1 resume o fluxo. O *Job* de carga faz requisições para o *Service*. O *Service* distribui as requisições entre os *pods* da aplicação. O Metrics Server fornece as métricas de CPU e o HPA altera o número de réplicas do *Deployment*.

Gerador de carga → Service web-hpa →
Deployment web-hpa (1–5 pods)
Metrics Server → HPA (CPU média, meta 50%)

Figura 1. Topologia lógica usada nas duas implantações.

A aplicação foi empacotada em uma imagem PHP/Apache. Ela expõe uma rota HTTP e aceita o parâmetro *work*, que executa um cálculo determinístico para elevar o uso de CPU. Cada contêiner solicita 100m de CPU e 64Mi de memória, com limites de 500m e 128Mi. Esses *requests* são importantes: a métrica de utilização do HPA é calculada em relação ao *request*, não ao limite.

O HPA foi configurado com mínimo de uma e máximo de cinco réplicas. Para subida, foi usada uma janela de estabilização de zero segundo e uma política de até 100% a cada 15 segundos. Para descida, foi usada janela de 60 segundos. Isso evita oscilações logo depois que a carga para.

III. IMPLANTAÇÃO LOCAL COM MINIKUBE

O ambiente local foi criado com Minikube usando o driver Docker. O Metrics Server foi habilitado como *addon*, a imagem foi construída localmente e carregada para o ambiente Minikube. Em seguida foram aplicados *namespace*, *Deployment*, *Service* e HPA.

Antes da carga, havia uma réplica em execução e a utilização observada era 1% frente à meta de 50%, como mostra a Figura 2. Durante a carga, o HPA detectou utilização acima da meta e aumentou as réplicas (Figura 3). Após remover o *Job*, a utilização voltou a um valor baixo (Figura 4). A permanência temporária de réplicas extras no pós-teste é esperada por causa da janela de estabilização.

```
Minikube / PSI5120 - ciclo continuo, antes da carga
2026-08-31T22:25:47-03:00
HPA: cpu 1%/50%, 1 replica; pod web-hpa-5cfbf7675-8vuw5 Running.
CPU observada: 1m; memoria: 13Mi.
```

Figura 2. Minikube antes da carga: uma réplica e CPU em 1% da referência do HPA.

```
Minikube / PSI5120 - ciclo continuo, durante a carga
2026-08-31T23:26:04-03:00
HPA: cpu 67%/50%, 2 replicas;
Pods Running: web-hpa-5cfbf7675-8vuw5, web-hpa-5cfbf7675-wjz24 e web-hpa-load-8z8bh.
CPU observada no pod original: 67%; memoria: 20Mi.
```

Figura 3. Minikube durante a carga, com HPA e pods adicionais.

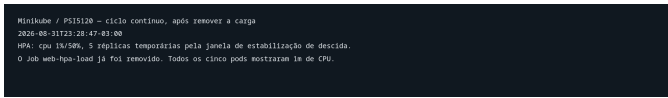


Figura 4. Minikube após remover a carga: CPU retorna a 1%; a redução de réplicas respeita a janela de estabilização.

IV. IMPLANTAÇÃO EM AMAZON EKS

Na nuvem, foi criado o cluster `psi5120-hpa` na região `sa-east-1` (São Paulo). A imagem da aplicação foi publicada no Amazon ECR e o *Deployment* do EKS referencia essa imagem. Foi utilizado um único *managed node group*, sem NAT Gateway, para reduzir o custo do experimento. O Metrics Server foi instalado como *add-on* do EKS [2].

Houve uma restrição relevante da conta: o tipo `t3.medium` foi rejeitado por não ser elegível ao Free Tier. O teste foi então executado com `t3.small`, que a própria conta listou como elegível. Esse desvio é relevante porque mostra que a escolha de máquina não pode ser assumida apenas pelo tutorial; ela depende das regras e créditos disponíveis na conta.

A Figura 5 registra o cluster ativo no Console da AWS, com Kubernetes 1.34 e região São Paulo. A infraestrutura foi mantida pequena e o *namespace* de teste foi removido logo após a coleta das evidências. Os *node groups* foram colocados em exclusão para evitar custo residual, como mostra a Figura 6.

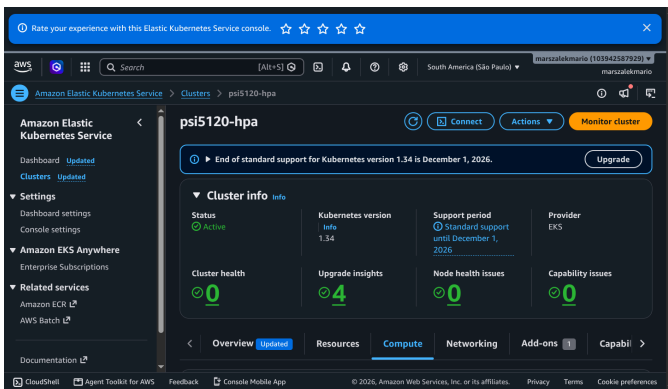


Figura 5. Console da AWS: cluster EKS ativo antes da limpeza.

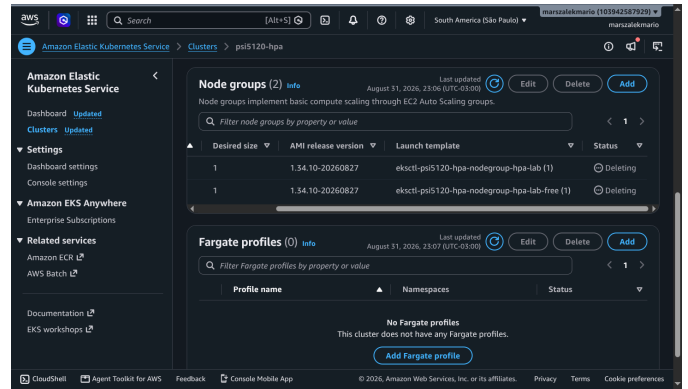


Figura 6. Console da AWS durante a limpeza: grupos de nós em exclusão após os testes.

V. METODOLOGIA DE TESTE

O teste seguiu três momentos: (i) confirmar o estado de repouso com `kubectl top pods` e `kubectl get hpa`; (ii) iniciar o *Job* de carga; e (iii) remover o *Job* e observar o retorno das métricas. As capturas foram feitas com os comandos `kubectl get hpa,pods` e `kubectl top pods`. Não foram usados valores estimados nas evidências: os números apresentados são os observados no cluster.

O *Job* `web-hpa-load` usa a imagem `busybox:1.36`. Seu comando inicia 20 processos concorrentes. Cada processo executa requisições HTTP com `wget` para o Service interno `web-hpa.psi5120-hpa.svc.cluster.local`, com o parâmetro `work=75000`, durante até 300 segundos. A carga fica dentro do cluster, portanto não depende de Load Balancer público nem de tráfego externo. Para encerrar o ensaio antes dos cinco minutos, o *Job* é removido explicitamente.

O parâmetro `work` é consumido pela aplicação PHP e provoca cálculo de CPU antes da resposta. O *Deployment* começa com uma réplica. Cada pod solicita 100m de CPU e 64Mi de memória, e tem limites de 500m e 128Mi. Como a meta do HPA é 50% de CPU, uma média próxima de 50m por pod já é suficiente para iniciar uma decisão de subida. A *readiness probe* HTTP usa atraso inicial de três segundos, período de cinco segundos e até seis falhas; assim, um novo pod só passa a receber tráfego depois de responder adequadamente.

O mesmo gerador de carga e a mesma política de HPA foram usados nas duas implantações. A diferença entre elas ficou restrita ao ambiente de execução e à origem da imagem: no Minikube a imagem foi carregada localmente, enquanto no EKS ela foi obtida do ECR. Isso reduz uma variável de confusão na comparação, pois a regra de escalonamento e o perfil de carga permaneceram os mesmos.

VI. DISPONIBILIDADE DO REPOSITÓRIO

O código-fonte, os manifestos, os roteiros de execução e as evidências usados neste trabalho estão disponíveis em <https://github.com/mmarsz/psi5120-hpa/>. O repositório contém o Dockerfile e a aplicação PHP, os manifestos de *namespace*,

Deployment, Service, HPA e Job de carga, além da configuração `eksctl` usada para o EKS.

As evidências estão separadas por ambiente. A pasta `minikube-hpa/evidence` contém as capturas do ciclo contínuo local, incluindo os estados antes, durante e após a carga. A pasta `minikube-hpa/evidence-eks` contém as capturas e transcritos da implantação gerenciada, incluindo o Console da AWS. Essa organização permite relacionar cada resultado discutido no artigo a um arquivo verificável.

O repositório também registra decisões necessárias para reproduzir o laboratório: imagem local no Minikube, imagem no ECR para EKS, alvo de CPU de 50%, limites de uma a cinco réplicas e ausência de NAT Gateway. O acesso pode ser liberado aos docentes para verificação da entrega, conforme solicitado no enunciado.

No EKS, antes da carga, a única réplica consumia 1m de CPU, isto é, 1% do `request` de referência (Figura 7). Durante a carga, o HPA apresentou 498%/50% e quatro réplicas ativas (Figura 8). Após remover o Job, a métrica retornou a 1%, enquanto a redução de réplicas respeitava a janela configurada (Figura 9). A carga também aparecia como um pod separado, consumindo 674m de CPU na observação do pico.

```

EKS / P515120 - evidencia antes da carga
2020-08-31T23:00:xx-03:00

$ kubectl -n p515120-hpa top pods
NAME                                CPU(cores)   MEMORY(bytes)
web-hpa-7cd8b054cb-x54nb            1m           139Mi

$ kubectl -n p515120-hpa get hpa
NAME      REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS
web-hpa   Deployment/web-hpa  cpu: 1%/50%  1         5         1

```

Figura 7. EKS antes da carga: HPA em 1%/50% e uma réplica.

```

EKS / P515120 - evidencia durante carga
2020-08-31T23:01:xx-03:00

$ kubectl -n p515120-hpa get hpa,pods -o wide
NAME      REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS
web-hpa   Deployment/web-hpa  cpu: 498%/50%  1         5         4

NAME      READY   STATUS    AGE
web-hpa-7cd8b054cb-gndux  1/1     Running   37s
web-hpa-7cd8b054cb-k0xsh  1/1     Running   22s
web-hpa-7cd8b054cb-rz15f  1/1     Running   22s
web-hpa-7cd8b054cb-x54nb  1/1     Running   2m08s
web-hpa-load-q201c        1/1     Running   65s

$ kubectl -n p515120-hpa top pods
NAME                                CPU(cores)   MEMORY(bytes)
web-hpa-7cd8b054cb-gndux           311m         159Mi
web-hpa-7cd8b054cb-k0xsh           75m          129Mi
web-hpa-7cd8b054cb-rz15f           136m         148Mi
web-hpa-7cd8b054cb-x54nb           351m         219Mi
web-hpa-load-q201c                 674m         79Mi

```

Figura 8. EKS durante a carga: HPA em 498%/50% e escalonamento em andamento.

```

EKS / P515120 - evidencia após remover a carga
2020-08-31T23:02:xx-03:00

$ kubectl -n p515120-hpa get hpa,pods -o wide
NAME      REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS
web-hpa   Deployment/web-hpa  cpu: 1%/50%  1         5         5

$ kubectl -n p515120-hpa top pods
NAME                                CPU(cores)   MEMORY(bytes)
web-hpa-7cd8b054cb-gndux            1m           159Mi
web-hpa-7cd8b054cb-k0xsh            1m           159Mi
web-hpa-7cd8b054cb-rz15f            1m           159Mi
web-hpa-7cd8b054cb-x54nb            1m           179Mi

Observação: a carga foi removida; as métricas retornaram a 1%. As 5 réplicas
permanecem temporariamente devido a janela de estabilização padrão do HPA.

```

Figura 9. EKS após remover a carga: CPU retorna a 1%; a descida respeita a janela do HPA.

VII. RESULTADOS E DISCUSSÃO

A Tabela I reúne os resultados práticos. A maior diferença está na operação. No Minikube, o ciclo de teste foi direto: criar o cluster, carregar a imagem e aplicar manifestos. No EKS, foi necessário configurar a imagem no ECR, criar o control plane, criar um grupo de nós, atualizar o `kubeconfig` e instalar o Metrics Server. Em troca, o EKS entrega um ambiente gerenciado e mais próximo do uso real em nuvem.

Tabela I
COMPARAÇÃO OBSERVADA ENTRE OS AMBIENTES

Item	Minikube	Amazon EKS
Execução	Local, com Docker e imagem carregada no cluster	Gerenciada, com ECR, EKS e nó EC2
Métrica	Metrics Server como add-on	Metrics Server como add-on EKS
Escala configurada	1 a 5 pods	1 a 5 pods
Evidência de pico	HPA e pods aumentados	498%/50% e 4 réplicas ativas no registro
Gestão	Menos etapas, ideal para desenvolvimento	Mais etapas, IAM e controle de custo

Sobre custo, não foi calculado um valor em reais neste artigo porque o laboratório usou créditos estudantis e teve execução curta. Ainda assim, o desenho tomou decisões para reduzir gasto: um nó pequeno, sem NAT Gateway, sem Load Balancer e remoção da aplicação após o teste. A criação de EKS não deve ser deixada ativa depois da entrega, pois o control plane e os recursos associados podem gerar cobrança.

VIII. TEMPO DE REAÇÃO E CUSTO OPERACIONAL

No Minikube foi registrado um ciclo contínuo com horário. Às 23:25:07, antes da carga, o HPA indicava 1%/50% e uma réplica. Entre essa observação e a leitura de 23:26:04, o Job foi iniciado; 57 segundos depois, a utilização estava em 67%/50% e o HPA já indicava duas réplicas. Esses registros não medem latência HTTP; eles medem o intervalo operacional entre iniciar a carga, coletar métricas e observar o escalonamento.

Após remover o Job, a leitura de 23:28:47 voltou a 1%/50%. As cinco réplicas ainda apareciam no instante dessa captura porque a política de descida usa janela de estabilização de 60 segundos. Portanto, a redução da métrica ocorreu antes da redução da quantidade de pods, comportamento compatível com a configuração do HPA.

No EKS, a captura de repouso mostra 1%/50% e uma réplica; durante a carga, mostra 498%/50% e quatro réplicas ativas. A infraestrutura em nuvem exigiu mais etapas que Minikube: imagem no ECR, criação do control plane, grupo de nós, atualização do `kubeconfig` e instalação do Metrics Server. Para limitar custo, foi usado um único nó `t3.small`, sem NAT Gateway e sem Load Balancer. Após os testes, o namespace foi removido, os grupos de nós foram colocados em exclusão e a exclusão do cluster foi iniciada.

IX. CONCLUSÃO

O objetivo foi atingido nas duas plataformas: a aplicação web foi implantada, o Metrics Server forneceu métricas e o HPA reagiu à carga de CPU. No EKS, a observação de 498% frente à meta de 50% e o aumento de uma para múltiplas réplicas deixam o comportamento visível. O retorno para 1% após remover a carga confirma que o gerador de carga era a causa do aumento.

Minikube foi mais simples para validar a lógica e ajustar manifestos. EKS exigiu mais trabalho de infraestrutura, especialmente credenciais, imagem no ECR e compatibilidade de tipo de instância, mas forneceu a validação em ambiente de nuvem gerenciada solicitada pelo enunciado. Como trabalho futuro, seria interessante repetir o ensaio com mais nós, medir latência de resposta e registrar custos detalhados por serviço.

APÊNDICE A

ROTEIRO DE REPRODUÇÃO

Este apêndice deixa o caminho usado no laboratório explícito. No Minikube, o cluster foi iniciado com driver Docker, quatro CPUs e 6GiB de memória. O Metrics Server foi habilitado, a imagem foi construída localmente e carregada no cluster. Depois foram aplicados `namespace.yaml`, `deployment.yaml`, `service.yaml` e `hpa.yaml`. A verificação inicial foi feita com `kubectl get deploy,pods,svc,hpa` e `kubectl top pods`.

No EKS, o arquivo `eksctl-cluster.yaml` definiu a região São Paulo, um nó gerenciado, acesso público ao endpoint e ausência de NAT Gateway. Após o cluster ficar ativo, o `kubeconfig` foi atualizado com `aws eks update-kubeconfig`. A imagem já publicada no ECR foi referenciada por `deployment-eks.yaml`. O Metrics Server foi instalado como add-on e a aplicação foi aplicada com os mesmos manifestos lógicos usados localmente.

O gerador de carga é um *Job* separado. Isso foi útil porque iniciar e encerrar a carga não exige alterar o *Deployment* principal. Durante o ensaio, os comandos de observação ficaram separados da aplicação: `kubectl get hpa`, `kubectl get pods` e `kubectl top pods`. No final, o *Job* foi removido com `kubectl delete job web-hpa-load`.

APÊNDICE B

LEITURA DAS EVIDÊNCIAS

As capturas não devem ser lidas como medição de desempenho absoluto da AWS. O objetivo do ensaio foi verificar a reação do HPA. O valor de 498%/50% significa que a utilização média calculada pelo HPA estava muito acima da meta configurada; por isso, o controlador aumentou as réplicas. As quatro réplicas ativas observadas na captura mostram que a ação não ocorre como uma troca atômica: pods precisam ser criados, agendados e passar pela *readiness probe*.

No pós-carga, a CPU retornou para aproximadamente 1m por pod. Ainda assim, o número de réplicas não caiu imediatamente. Isso é comportamento esperado e foi configurado de propósito: a janela de estabilização de descida de 60 segundos

impede que pequenas oscilações de tráfego façam o sistema escalar para cima e para baixo o tempo todo.

APÊNDICE C

LIMITAÇÕES E DECISÕES

O teste usa uma única máquina no EKS. Logo, ele prova escalonamento de pods, não escalonamento de nós. Quando o HPA pede mais pods do que cabem no nó, novos pods podem esperar por capacidade. Em um ambiente de produção, seria possível combinar HPA com Cluster Autoscaler ou Karpenter e definir mais de um nó. Isso não foi feito porque o foco do trabalho era HPA e porque o laboratório tinha orçamento limitado.

Também não foi criado Load Balancer público. O Service é do tipo ClusterIP e o tráfego de teste vem de dentro do cluster. Essa escolha reduz custo e deixa o experimento reproduzível sem expor a aplicação à Internet. Por fim, o valor financeiro não foi estimado como se fosse uma medição precisa; o artigo registra apenas as escolhas de contenção de custo e a limpeza iniciada depois das evidências.

APÊNDICE D

CRONOLOGIA DO EXPERIMENTO EM EKS

A execução em nuvem foi feita em etapas para evitar confundir falha de infraestrutura com falha do HPA. Primeiro, o control plane foi criado e sua situação foi conferida como *Active*. Segundo, o nó gerenciado foi criado e confirmado como *Ready* no Kubernetes. Terceiro, a imagem no ECR, o Metrics Server e os manifestos foram validados. Somente então o *Job* de carga foi iniciado.

Durante a criação do nó ocorreu uma rejeição de tipo de instância pelo Free Tier. A tentativa foi interrompida, o grupo em erro foi colocado em exclusão e foi usado um novo grupo com `t3.small`. Essa sequência foi preservada nas evidências do Console de limpeza. O ponto importante é que o teste final não dependeu de um recurso em estado de erro: a aplicação foi executada em um nó que apareceu como *Ready* e o cluster foi observado ativo pelo Console e pelo `kubectl`.

O HPA ficou inicialmente com uma réplica. O primeiro ciclo de métricas reportou 1%/50% e, no pico registrado, 498%/50%. Esse intervalo ilustra a natureza periódica do controlador: a escala é decidida depois de coletar métricas e os novos pods levam algum tempo para ficarem disponíveis. Por isso, o artigo usa as observações em sequência e não um único número isolado.

APÊNDICE E

RASTREABILIDADE DOS ARQUIVOS

O repositório foi organizado para que a correção não dependa de comandos escondidos. O `Dockerfile` e `app/index.php` definem a aplicação. A pasta `manifests` contém os recursos Kubernetes; há uma versão local do *Deployment* e outra para EKS, que troca apenas a origem da imagem pelo ECR. O arquivo `eksctl-cluster.yaml` registra a topologia do cluster gerenciado e `cleanup-eks.sh` concentra a remoção da infraestrutura.

As imagens e os respectivos transcritos de terminal ficam em duas pastas. `evidence` contém o Minikube, e `evidence-eks` contém o EKS. Os arquivos são nomeados pelo instante do teste; as referências no texto usam a numeração gerada pelo LaTeX. A captura do Console do cluster ativo está na Figura 5. Essa separação permite que cada afirmação do texto seja conferida em um artefato do repositório.

Para reproduzir o experimento, deve-se manter os comentários dos manifestos e ajustar apenas credenciais, nome de imagem no ECR e dados da conta. Nenhuma credencial é armazenada no repositório. Esse cuidado é especialmente importante porque o trabalho pede um link que pode ser compartilhado com docentes.

APÊNDICE F

AMEAÇAS À VALIDADE DO EXPERIMENTO

Os resultados devem ser interpretados como validação funcional do HPA, e não como *benchmark* de desempenho entre plataformas. Minikube e EKS foram executados em recursos diferentes, com camadas de virtualização, rede e disponibilidade de CPU distintas. Por esse motivo, percentuais de CPU e quantidade de réplicas comprovam a reação do controlador em cada ambiente, mas não permitem concluir que uma plataforma é mais rápida que a outra.

O experimento no EKS usou apenas um nó `t3.small`. Assim, o HPA pode solicitar novos pods sem que isso implique criação automática de novos nós. Essa decisão foi deliberada para manter o escopo no autoescalamento de pods e limitar o uso de créditos estudantis. Em uma implantação de produção, a ausência de capacidade no nó precisaria ser tratada por um mecanismo complementar de escalonamento de nós.

Por fim, as métricas são amostras do Metrics Server e do ciclo de reconciliação do HPA. A captura durante a carga registra um estado observado, não uma série temporal completa. Mesmo com essa limitação, as evidências mostram a sequência essencial: repouso com uma réplica, CPU acima da meta com novas réplicas e retorno da CPU a 1% depois da remoção do Job.

REFERÊNCIAS

- [1] Kubernetes, “Horizontal Pod Autoscaling Walkthrough.” Disponível em: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>. Acesso em: 31 ago. 2026.
- [2] Amazon Web Services, “Scale pod deployments with Horizontal Pod Autoscaler.” Disponível em: <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>. Acesso em: 31 ago. 2026.